Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- Facade kalıbı (yapısal)
  - Amaç (ekrandan, 00:07:40): çeşitli alt sistemlerde bulunan farklı arayüzleri tek ve daha üst düzey bir arayüzde toplamak. Böylece istekçiler alt sistemlerle ilgili alt düzey ayrıntılardan soyutlanır.
  - Adı binaların ön yüz kaplamasından (cephe) gelir: sisteme bir ön yüz sunar.
  - Kullanım: karmaşık ve iyi tasarlanmamış bir alt sistemde yapılacak işler birçok sınıfın birçok metoduna dağılmıştır. Facade sınıfı, sistemin temel işlevlerini başlatan metotları sunar ve bu metotlar ilgili sınıfların metotlarını çağırır.
  - Hoca kalıbı basit buldu, MVC ve Adapter'a benzeyen bir fikir olarak niteledi ve ayrı örnek yapmadı.
- Flyweight kalıbı (yapısal)
  - Amaç (ekrandan, 00:10:00): paylaşım yolu ile çok sayıda basit nesnenin kullanımını desteklemek. Türkçede "tüy sıklet" ya da "sinek sıklet" diye çevrilebilir.
  - Örnek: metin düzenleme programında her harfi en iyi görüntü için bir grafik nesnesi olarak ele almak. Her harf için ayrı nesne tutulursa küçük bir belgede bile binlerce nesne olur; bellekte ve ikincil saklama ortamında kaplanan yer aşırı olur. Hocanın örneği: dersin üç sayfalık öneri formunda boşluklarla birlikte 5000 kadar harf vardır.
  - Çözüm: Unicode kümesindeki her harften bellekte tek nesne içeren bir havuz tutulur; metin içinde yalnız bu nesnelere işaretçi (ya da harf adını anahtar olarak kullanan bir map ile erişim) ve satır-sütun bilgisi saklanır.
  - Metin yapısı (ekrandan, 00:15:30): soyut `Glyph {Draw(Context), Intersects(Point, Context)}`; alt sınıflar `Row`, `Character` (`char c` alanı) ve `Column`. `Row` ve `Column` birer `children` topluluğu ile `Glyph` içerir. Composite'e benzer, ama burada parça-bütün ilişkisinde nesnelerin kendileri değil flyweight işaretçileri yer alır.
  - Kalıp yapısı (ekrandan, 00:20:00): `FlyweightFactory {GetFlyweight(key)}` bir `flyweights` havuzu tutar. Not kutusu: `if (flyweight[key] exists) { return existing flyweight; } else { create new flyweight; add it to pool of flyweights; return the new flyweight; }`. Soyut `Flyweight {Operation(extrinsicState)}`; alt sınıflar `ConcreteFlyweight {Operation(extrinsicState), intrinsicState}` ve `UnsharedConcreteFlyweight {Operation(extrinsicState), allState}`. `Client`, fabrikayı ve flyweight'leri kullanır.
  - Anahtardan yola çıkarak flyweight oluşturulamıyorsa `GetFlyweight` null döndürecek biçimde kodlanabilir. Ortak üst türde somut metot ya da alan yoksa arayüz (interface) kullanılabilir.
  - Durum bilgisi ikiye ayrılır: içsel (intrinsic, değişmez, immutable) ve dışsal (extrinsic, değişebilir, mutable). Metin örneğinde içsel durum harfin büyük çizim verisi, dışsal durum harfin satır-sütun konumudur. İçsel kısım isterseniz `final` tanımlanabilir.
  - Kullanım koşullarının tümü gerekir: uygulama çok sayıda nesne kullanır; bu yüzden saklama masrafı yüksektir; nesne durumunun büyük kısmı dışsallaştırılabilir; dışsal durum ayrılınca çok sayıda nesnenin yerine az sayıda paylaşılan nesne kullanılabilir; uygulama nesne kimliğine (object identity) bağımlı değildir.
- Flyweight örneği: kimya fabrikası (Design Patterns Java Workbook'tan uyarlanmış)
  - İlk çözüm (ekrandan, 00:30:00): `KimyasalMadde` (`gram`, `isim`, `simge`, `atomAgirligi`; `getMol()` gövdesi `return gram/atomAgirligi;`) ve `Karışım` (`isim`, `maddeler : LinkedList<KimyasalMadde>`, `maddeEkle(KimyasalMadde)`, `tarifEt()`). Örnek: 100 gr ba-

rutta 75 gr potasyum nitrat, 15 gr karbon, 10 gr kükürt bulunur.
  - Sorun: karbon başka karışımlarda başka gramajla bulunur; yalnız gram bilgisi farklı, kalan durum bilgisi aynı olan çok sayıda karbon nesnesi oluşur. Bellekten çok bakım sorun olur: simge yanlış yazılırsa hangi kopyaların düzeltileceği belli olmaz.
  - İkinci çözüm: değişen gram bilgisi ayrı bir `Bileşen` sınıfına alınır. Karışım bileşenler içerir, her bileşen bir kimyasal maddeyle ilişkilidir.
  - Flyweight çözümü (ekrandan, 00:40:00): `Bileşen {gram, madde; getGram(), getMol(), getMaddeAdi()}`, `KimyasalMadde {isim, simge, atomAgirligi; getMol(gram)}`, `Karışım.maddeEkle(simge : String, gram : double)` ve `MaddeFabrikası {maddeler : HashMap<String, KimyasalMadde> {static}; maddeBul(simge) {static}}`.
  - `MaddeFabrikasi` kodu (ekrandan, 00:44:40): tablo `static { ... }` bloğunda doldurulur: `maddeler.put("C", new KimyasalMadde("Karbon", "C", 12.0107))`, `maddeler.put("S", new KimyasalMadde("Kükürt", "S", 32.066))`, ayrıca `"KNO3"` (Potasyum Nitrat) ve `"NaNO3"` (Sodyum Nitrat). `maddeBul(simge)` gövdesi `return maddeler.get(simge);`.
  - `Karisim.maddeEkle(simge, gram)` (ekrandan, 00:46:50): `maddeler.add(new Bilesen(gram, MaddeFabrikasi.maddeBul(simge)))`. `tarifEt()` her bileşen için gram ve madde adını bir metne ekler.
  - Ana program (ekrandan, 00:50:00) “Kara barut” (NaNO3 75, C 15, S 10) ve “Temiz barut” (KNO3 75, C 20, S 25) karışımlarını oluşturup tariflerini yazdırır.
- Proxy kalıbı (yapısal)
  - Amaç: bir nesneye erişimi denetlemek için onun yerine geçecek bir vekil nesneyi araya koymak.
  - Örnek: çok sayıda büyük resim içeren bir belgeyi açmak ve kaydırmak yavaştır. Belge editörü resmi değil, resmin dosya yolunu tutan bir resim vekilini işaret eder; vekil resmi gerektiğinde yükler.
  - Örnek çözüm (ekrandan, 01:10:00): `DocumentEditor` → soyut `Graphic {Draw(), GetExtent(), Store(), Load()}`; alt sınıflar `Image {imageImp, extent}` ve `ImageProxy {fileName, extent}`. `ImageProxy.Draw()`: `if (image == 0) { image = LoadImage(fileName); } image->Draw();`. `GetExtent()`: `if (image == 0) { return extent; } else { return image->GetExtent(); }`.
  - Kalıp yapısı (ekrandan, 01:11:40): `Client` → soyut `Subject {Request()}`; alt sınıflar `RealSubject {Request()}` ve `Proxy {Request()}`; Proxy, `realSubject` üyesine sahiptir ve `Request()` içinde `realSubject->Request()` çağırır. Çalışma anında: `aClient.subject` → `aProxy`, `aProxy.realSubject` → `aRealSubject`.
  - Kullanım: vekil ile asıl nesne farklı adres uzaylarında olabilir (dağıtılmış sistemler, Java RMI). Kopyalanan nesnelerde tutarlılık denetimi, önbellek, yenileme gibi ayrıntıları istekçiden gizler. Erişim izni denetleyen vekile koruma vekili (protection proxy) denir.
- Proxy örneği: izle-öde (pay-per-view) yayın sistemi
  - Elde hazır, ücretsiz yayın yapan bir sunucu vardır; araya ödeme denetimi yapan bir vekil konur.
  - Yapı (ekrandan, 01:20:00): `«interface» StreamSource {stream(programID, destination : String) : boolean}`; `StreamServer` ve `StreamProxy` bu arayüzü gerçekler. `StreamProxy {original : StreamSource; StreamProxy(original), stream(...), checkPayment(programID, destination) : boolean}`. `StreamDestination {server : StreamSource, address : String; getAddress(), setServer(StreamSource), makePayment(double), requestProgram(programID)}`.
  - `StreamProxy.stream()` (ekrandan, 01:23:10): `if (checkPayment(programID, destination)) return original.stream(programID, destination); else return false;`.
  - İlişkiler: vekile asıl sunucu, istemcilere (destination) vekil verilir; istemci kiminle konuştuğunu bilmez.
- Davranışsal (behavioral) kalıplara geçiş ve Chain of Responsibility (Sorumluluk Zinciri)
  - Amaç: ilişkili bir nesneler zinciri ya da hiyerarşisinde bir nesnenin, aldığı mesajı en doğru biçimde işleyemeyeceği durumlarda mesajı bu işi daha iyi yapabilecek başka bir nesneye iletmesi. Nesnede o metot vardır; sorun metodun olmaması değil, daha doğru yanıtı başka bir nesnenin verebilmesidir.
  - Örnek: diyalog kutusunda bağlama duyarlı yardım. Bir bileşene özel yardım yoksa istek sırayla içinde bulunduğu panele, diyaloga ve en sonunda uygulamaya iletilir

(özelden genele).
  - Çözüm yapısı (ekrandan, 01:34:10): soyut `HelpHandler {HandleHelp()}` kendine `handler` adlı ilişkiyle bağlıdır; varsayılan gövde `handler->HandleHelp()`. Alt sınıflar `Application` ve soyut `Widget`; `Widget` altında `Dialog` ve `Button {HandleHelp(), ShowHelp()}`. `Button.HandleHelp(): if can handle { ShowHelp() } else { Handler::HandleHelp() }`.
  - Kullanım: bir mesajı yanıtlayabilecek birden fazla nesne varsa ve hangisinin yanıt vereceği önceden belirlenemiyorsa ya da çalışma anında belirlenmesi tercih ediliyorsa.
- Chain of Responsibility örneği: reklam yönetim yazılımı (hocanın kendi örneği)
  - Problem (ekrandan, 01:40:00): Kampanya 1: spor sitesinin 18-39 yaş arası erkek ziyaretçilerine traş bıçağı, 18-39 yaş arası kadın ziyaretçilerine fitness salonu, diğer ziyaretçilere spor mağazasındaki indirimin reklamı. Kampanya 2: haber sitesinin 18-29 yaş arası A gelir grubu erkek ziyaretçilerine otomobil dergisi reklamı, aksi hâlde kampanya 1. Kampanya 3: coğrafi bölge ve gelir düzeyine göre lokanta reklamı.
  - Sınıflar: `Cinsiyet` (alanlı ve kuruculu enum: `KADIN("Kadın"), ERKEK("Erkek")`, ekrandan, 01:50:00), `GelirDuzeyi` (`A, B, C`), `User` (cinsiyet, gelir düzeyi, yaş ve `yasDahilMi(alt, ust)`), `Banner`, soyut `Reklam` ve alt sınıfları `TrasBicak`, `FitnessSalonu`, `IndirimSporMagaza`, `OtoDergi`.
  - `Reklam` (ekrandan, 01:51:10): `banner` ve `sonraki` alanları; soyut `reklamUygunMu(User)` ve `reklamGoster(User)`; `public final void reklamKotar(User u)`: uygunsa sınıf adını ve kullanıcıyı yazıp `reklamGoster(u)`, değilse ve `sonraki != null` ise iletimi yazıp `sonraki.reklamKotar(u)`.
  - `TrasBicak.reklamUygunMu` (ekrandan, 01:55:30): `if (u.getCinsiyet() == Cinsiyet.ERKEK) if (u.yasDahilMi(18, 39)) return true; return false;`. `IndirimSporMagaza` her kullanıcı için `true` döndürür ve zincirin sonuna konur.
  - Ana program (ekrandan, 02:00:00): kullanıcılar (ERKEK, C, 23), (ERKEK, A, 53), (KADIN, B, 33); zincir TrasBicak → FitnessSalonu → IndirimSporMagaza. Sonuç: ilk kullanıcıya traş bıçağı, ikincisine spor mağazası indirimi, üçüncüsüne fitness salonu reklamı gösterildi.
- Command kalıbı (davranışsal)
  - Amaç: nesneye giden istekleri (metot çağrılarını) birer nesneye çevirmek. Böylece çağrılar kuyruğa alınabilir ya da günlüğü (log) tutulabilir.
  - Örnek: GUI'de menü seçeneğine tıklanınca yapılacak iş doğrudan tıklama metoduna yazılmaz; `Command` arayüzündeki `execute()` metoduna sahip bir nesne verilir. Dosya açma (OpenCommand) ve yapıştırma (PasteCommand) birer komut alt sınıfı olabilir.
  - Kalıp yapısı (ekrandan, 02:20:00): `Invoker` bir `Command {Execute()}` tutar; `ConcreteCommand {Execute(), state}` Command'dan türer ve `receiver` üyesiyle `Receiver {Action()}`'a bağlıdır; `Execute()` gövdesi `receiver->Action();`. `Client`, Receiver ve ConcreteCommand'ı oluşturur. Receiver isteğe bağlıdır.
- Command örneği: akıllı ev kumandası (Head First Design Patterns'tan esinlenilmiş)
  - Yapı (ekrandan, 02:23:40): «`interface`» `AcKapa {ac() : boolean, kapa() : boolean}`; `AydinlatmaDugmesi`, `ElektronikAlet`, `Iklimlendirme` (her biri `isim` alanı, `ac(), kapa(), toString()`); `Kumanda {acKapaDugmeleri : ArrayList<AcKapa>; ac(dugmeNo), kapa(dugmeNo), dugmeEkle(AcKapa), hepsiniKapa()}`; `AnaProgram {kumanda; evBoskenSokakKapisiAcildi(), evdeKimseKalmadi(), main}`.
  - `Kumanda.ac(dugmeNo)`: ilgili düğme yoksa `false`; varsa listeden alıp `ac()` çağırır ve onun sonucunu döndürür.
  - Senaryolar (ekrandan, 02:30:00): `evBoskenSokakKapisiAcildi()` → `kumanda.ac(0); kumanda.ac(5);` (antre ışığı ve kombi), `evdeKimseKalmadi()` → `kumanda.hepsiniKapa();`.

Hocanın Özellikle Vurguladığı Kısımlar

- Flyweight'in tasarruf sağlaması için durum bilgisi içsel ve dışsal olarak ikiye ayrılabilmelidir
  - Kazanç, içsel bilginin hacimli, dışsal bilginin hafif olmasıyla orantılıdır. İkisinin boyutu birbirine yaklaşıyorsa kalıbı kullanmaya gerek yoktur.
- Flyweight nesne kimliğini kaybettirir
  - Hava yolu örneği: ayda bir milyon bilet satılması, bir milyon farklı yolcu demek değildir. Kişi başına ortalama iki uçuşta tekil yolcu sayısı 500.000'dir. Tekil kişiler önemliyse (her kişinin kendi kimliği olmalıysa) Flyweight kullanılamaz.

- Flyweight ile Singleton farkı
  - Flyweight'te de bir harften tek nesne vardır, ama ona işaret eden ve dışsal durumu taşıyan çok sayıda işaretçi vardır. Bu fark gözden kaçırılmamalıdır.
- "Neyin değiştiğini bul ve onu sarmala"
  - Kimya örneğinde değişen şey gramajdır; ayrı bir sınıfa (Bileşen) alınmalıdır. İyi bir nesneye dayalı modelleme ile Flyweight arasındaki tek fark ek bir fabrika sınıfıdır. Hoca bunu geleceğe yönelik küçük bir altyapı yatırımı olarak niteledi: karışım sayısı ve aynı maddenin geçme sıklığı arttıkça tasarruf artar.
- Composite neden kullanılmadı
  - Karışımda hiyerarşik bir parça-bütün ilişkisi yoktur; maddeler aynı düzeyde eklenir. Ara birleşimler sırayla oluşturulup kullanılsaydı Composite düşünülebilirdi.
- Proxy, vekili olduğu nesneyle aynı arayüze sahip olmalıdır
  - İstemci vekille mi asıl nesneyle mi konuştuğunu bilmemelidir; bu ancak ortak bir soyut sınıf ya da arayüzle sağlanır.
- Proxy ile Decorator farkı
  - Proxy'de zincir yalnız iki öğelidir (vekil ve asıl nesne). Decorator'da üç, dört, beş öğeli zincirler doğal bir ihtiyaçtır.
- Chain of Responsibility'de zincir doğru kurulmalı ve doğru sonlandırılmalıdır
  - Aksi hâlde çağrı boşluğa düşer; kullanıcı tıklar, hiçbir şey olmaz. En azından "yardım dosyası bulunamadı" gibi bir tepki verilmelidir.
  - Zinciri kurma ve izleme işi, alt sınıflarda if-else ile sınıf adlarına bakmak yerine soyut üst sınıfa somut metotlarla konmalıdır.
  - Reklam örneğinde her zaman `true` döndüren reklam (spor mağazası indirimi) zincirin sonuna konur. Bu `return true` bir iskelet kod değil, iş mantığının kendisidir.
- Command'da receiver isteğe bağlıdır
  - Her komutun bir alıcısı olmak zorunda değildir, ama alıcı desteği kalıbın önemli bir yeteneğidir.

Kısa Tekrar Notları

- Facade: karmaşık alt sisteme tek, üst düzey bir ön yüz; istekçi alt düzey sınıflarla uğraşmaz.
- Flyweight: FlyweightFactory + havuz + paylaşılan ConcreteFlyweight; anahtar varsa var olanı, yoksa yenisini oluşturup havuza ekleyip döndür.
- İçsel durum = paylaşılan, değişmez (immutable); dışsal durum = istemcinin hesaplayıp verdiği, değişen (mutable).
- Flyweight nesneleri fabrikanın private iç sınıfı yapılırsa yalnız havuz üzerinden erişilebilir.
- Java'da `static { ... }` bloğu sınıf belleğe yüklenince bir kez çalışır; static alanları ilklendirmek için kullanılabilir.
- Proxy: Subject / RealSubject / Proxy; vekil gerektiğinde asıl nesneyi yükler, çağrıyı ona yönlendirir, önüne ya da arkasına iş ekleyebilir.
- Proxy türleri: uzak vekil (remote, ör. Java RMI), sanal vekil (resmi geç yükleme), koruma vekili (ödeme ya da erişim denetimi).
- Chain of Responsibility: Handler'ın kendine (sonraki) ilişkisi vardır; işleyemeyen nesne isteği sonrakine iletir.
- Smalltalk'ta tanımsız bir metot çağrılınca derleme hatası yerine `doesNotUnderstand` metodu devreye girer. Chain of Responsibility bununla aynı şey değildir: zincirdeki nesnede metot tanımlıdır.
- Java enum'ları alan, kurucu ve metot içerebilir.
- Command: Client / Invoker / Command / ConcreteCommand / Receiver; `Execute()` → `receiver->Action()`.

Detaylı Açıklamalar

- Flyweight'in metin editörü örneği
  - Her harf yüksek çözünürlüklü bir grafik nesnesi olursa nesneyi dosyada aynen saklamak dosya boyunu büyütür. Dosyada yalnız harf adı (HTML ve CSS gibi bir biçimde) saklansa bile belge belleğe yüklenince her harfin resmi de yüklenmek zorunda kalır.

  - Sabit genişlikli bir fontta (daktilo gibi) satır ve sütun sayısı kağıt boyutuna göre bellidir. Değişken genişlikli fontta da "3. satırdaki 5. karakter" biçiminde konum tutulabilir.
  - Hocanın sayısal örneğinde 5000 karakterin boşluklar çıkınca yaklaşık 4000'i harftir. Her harf için megabaytlarca veri yerine 4 ya da 8 baytlık bir işaretçi tutulur.
  - Hocaya göre "hafif" olan nesnenin kendisi değil, ona verilen referanstır: nesneler büyük, onlara ulaşan işaretçiler küçüktür.
  - Satırlar kökte, sütunlar ara düğümlerde, karakterler yapraklarda olabilir ya da tersi; yapı iki düzene de izin verir. Bir satır çizilirken içindeki bütün harfler çizilir; `Character` kendi kodunu anahtar yaparak havuzdan ilgili nesneyi alır.
  - Paylaşılması gerekmeyen flyweight'ler için havuzda yer almayan bir alt sınıf (UnsharedConcreteFlyweight) da olabilir, ama ne kadar çok paylaşım varsa o kadar tasarruf vardır.
  - Hoca, slayttaki şemada istemcilerle fabrika arasında da ok çizilmesinin yararlı olacağını söyledi: istemci flyweight'e ancak fabrikadan isteyerek ulaşır.
- Kimya fabrikası örneğinin karşılaştırması
  - İlk çözümde `KimyasalMadde` kurucusu bütün alanları alır, bütün alanlar için getter vardır; setter yalnız gram içindir, çünkü simge ve atom ağırlığı asla değişmez. `getMol()` mol sayısını `gram/atomAgirligi` olarak hesaplar.
  - Bin karışım ve madde başına bir kilobayt varsayılsa bile toplam bir megabayt eder; hoca asıl sorunu bellekten çok bakım olarak gördü.
  - Flyweight çözümünde `maddeEkle` maddenin kendisini değil, tekil tanımlayıcısı olan kimyasal simgeyi alır. Fabrika bir tür kayıt defteri (registry) gibi davranır ve üretilen maddelerin kataloğunu tutar.
  - Fabrikanın tablosu ve `maddeBul` metodu kolaylık için static tanımlandı; karbon her yerde karbondur, farklı fabrikalara gerek yoktur. Farklı fabrika nesneleri gerekseydi static yapılmazdı.
  - Kullanılan dil ya da optimize edici zaten yalnız referans saklıyorsa Flyweight'in alt düzey kazancı az olur. Hocaya göre kalıplarla düşünmek doğru çözüme daha kısa sürede ulaştırır.
  - Hoca, slayttaki şemada `Karışım` sınıfının `maddeler` alanının `LinkedList<KimyasalMadde>` yazıldığını, kodda olduğu gibi `LinkedList<Bilesen>` olması gerektiğini söyledi; alanın adının da "bileşenler" olmasının daha doğru olacağını not etti.
  - Ders sonrası not: Flyweight çözümü slaytında (ekrandan, 00:40:00) `KimyasalMadde.getIsim()` dönüş türü `double` yazılmıştır; isim bir `String` olduğu için doğrusu `String` dönüşüdür (İlk çözüm slaytında da aynı yazım vardır).
  -
- Proxy örneğinin ayrıntıları
  - Resim vekili, resmin boyutunu (extent) belgeyi oluştururken öğrenip saklayabilir; böylece boyut için resmi yüklemek gerekmez. Resim yüklenmişse boyut resmin kendisinden alınır.
  - Vekil, asıl nesneden önce devreye girer, gerekirse onu yükler, sonra asıl işi ona yaptırır. Decorator gibi işin başına ve sonuna ek işler koyabilir.
  - Dağıtılmış sistemlerde başka bir makinedeki vekil, ağdaki nesneyle bağlantıyı kurar ve istemciye sanki nesne yereldeymiş gibi hizmet verir. Nesne erişimi hızlandırmak için birkaç düğüme kopyalanırsa asıl nesnedeki değişiklik bütün kopyalara yansıtılmalıdır; bunun için Observer kalıbı kullanılabilir.
  - Java RMI (Remote Method Invocation), başka bir bilgisayardaki sanal makinede çalışan bir nesneye erişimi dil ile birlikte gelen bir altyapıyla sağlar. Mikroservisler ve bulut yapıları da dağıtılmış sistemlerin deneyimleri üzerine kuruludur.
  - Örnekte `StreamServer.stream()` gövdesi yazılmamıştır; `checkPayment` de gerçek bir ödeme denetimi yerine sabit bir değer döndürür. Ödeme yoksa ekranda "lütfen ödeme yapın" diyaloğu gösterilebilir.
- Chain of Responsibility örneğinin ayrıntıları
  - Yaş aralığı denetimi `User` sınıfına kondu, çünkü bilgi kullanıcıdadır. `yasDahilMi(alt, ust)` alt ve üst sınır dahil olmak üzere aralıktaysa `true` döndürür.
  - `Reklam` soyut sınıftır: kurucusu somut kodlanabilir, ama uygunluk kuralı her reklamda farklıdır; gösterme işi de reklam türüne göre değişir (bitmap, HTML5 animasyonu, video). Bu yüzden `reklamUygunMu` ve `reklamGoster` soyuttur.
  - Ekrana yazılan iz, zincirin doğru çalıştığını gösterir: `TrasBicak->Erkek,C,23`; ikinci

kullanıcı için `TrasBicak->FitnessSalonu`, `FitnessSalonu->IndirimSporMagaza`, `IndirimSporMagaza->Erkek,A,53`; üçüncü kullanıcı için `TrasBicak->FitnessSalonu`, `FitnessSalonu->Kadın,B,33` (ekrandan, 02:00:00, paket adı kısaltıldı).
  - Hoca, zincirin sonlanmasını üst sınıfta denetlemek yerine her zaman uygun olan bir reklamı zincire koymayı tercih etti. En fazla, son `else` kolunda bir çalışma zamanı istisnası (runtime exception) fırlatılabilir.
  - Ana programda dizi ters sırada doldurulduğu: önce `reklamlar[2] = new IndirimSporMagaza(new Banner(), null)`, sonra `reklamlar[1] = new FitnessSalonu(new Banner(), reklamlar[2])`, sonra `reklamlar[0] = new TrasBicak(new Banner(), reklamlar[1])`. Böylece her reklam oluşturulurken sonraki reklamı hazırdır.
  - Yeni bir kampanya için yalnız yeni bir `Reklam` alt sınıfı ve onun uygunluk kuralı yazılır; zincir altyapısı üst sınıftan gelir.
- Command örneğinin ayrıntıları
  - Cihaz sınıflarının `ac()` ve `kapa()` gövdeleri aynıdır: durumunu ve adını yazıp `true` döndürür. Gerçek bir iklimlendirme cihazında açılış adımları daha karmaşık olur.
  - Ana programın kurucusu evdeki bileşenleri kumandaya ekler: antre, salon, mutfak ve oda aydınlatma düğmeleri, bir televizyon (elektronik alet) ve bir kombi (iklimlendirme). 0 numaralı düğme antre ışığı, 5 numaralı düğme kombidir.
  - Çıktıda (ekrandan, 02:30:00) “Evden çıkılıyor” satırından sonra bütün bileşenler `KAPALI: AcKapa:Antre`, `Salon`, `Mutfak`, `Oda`, `Vestel TV`, `Kombi` biçiminde kendini kapatır.
  - Hoca bu örnekten tam memnun olmadığını, GoF’taki dosya menüsü örneğinin daha iyi olduğunu söyledi.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.